

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**федеральное государственное бюджетное образовательное учреждение
высшего образования**

**«Российский государственный университет им. А.Н. Косыгина
(Технологии. Дизайн. Искусство)»**

**Кафедра автоматизированных систем обработки информации и
управления**

**Отчет
по курсовой работе
по дисциплине «Проектирование баз данных»**

Выполнил: Ольховский Н. С. гр. ИТА-123

Принял: Монахов В. И.

Москва 2026

Задание

Проведение инвентаризации готовой продукции на складе

Предприятие производит готовую продукцию. Готовая продукция характеризуется: артикулом, наименованием, размером, ед.измерения, принадлежностью к определенной группе (код и наименование).

Готовая продукция хранится на складе. Данные об остатках готовой продукции хранятся в книге остатков (артикул, размер, сорт, количество по книжным остаткам).

Периодически проводится инвентаризация готовой продукции (опись фактического наличия продукции). Данные инвентаризации фиксируются в книгу инвентаризации, в которой записываются: дата инвентаризации, номер акта, а также ассортимент (артикул, размер, сорт, фактическое количество). Так как одна и та же продукция может храниться в разных местах на складе, то ассортимент (артикул, размер, сорт) может повторяться.

По данным инвентаризации необходимо сформировать сличительную ведомость по продукции заданной группы: наименование продукции, фактический и книжный остаток, излишек (недостача).

Содержание

Введение.....	4
1. Описание логической модели данных	5
2. Описание физической модели данных.....	9
3. Создание базы данных.....	12
Заключение	18
Список литературы	19
Приложение 1. Связи между сущностями.....	20
Приложение 2. Текст скрипт-файла создания схемы БД.....	21
Приложение 3. Тексты файлов для загрузки данных таблиц.....	23
Приложение 4. Скрипты команд загрузки и выгрузки данных.....	25

Введение

В современном мире эффективное управление предприятием невозможно без использования автоматизированных информационных систем. Информационные системы играют ключевую роль в сборе, хранении, обработке и анализе данных, обеспечивая принятие обоснованных управленческих решений. Центральным звеном любой такой системы является база данных, которая выступает в роли единого хранилища структурированной информации, обеспечивающей интеграцию всех бизнес-процессов компании.

Особое значение информационные системы имеют для управления материальными запасами и контроля за сохранностью товарно-материальных ценностей. Важнейшим элементом этого контроля является инвентаризация, позволяющая выявить фактическое наличие ценностей и сопоставить его с учетными данными. Автоматизация процессов учета готовой продукции и проведения инвентаризаций позволяет существенно снизить трудоемкость, минимизировать риск ошибок и своевременно выявлять расхождения.

В ходе выполнения курсовой работы были решены задачи анализа предметной области, разработки логической и физической модели данных, спроектирована и создана реляционная база данных в СУБД PostgreSQL.

1. Описание логической модели данных

В ходе проектирования базы данных для задачи учёта готовой продукции и проведения инвентаризации была разработана логическая модель данных, отражающая структуру предметной области на уровне сущностей, их атрибутов и связей между ними. Модель построена с использованием CASE-средства ERwin 4 [1] и представлена в двух нотациях: диаграмма «сущность–связь» (ER-диаграмма) в нотации IE (Information Engineering) на рисунке 1 и полная атрибутивная модель в нотации IDEF1X на рисунке 2, на которой для каждой сущности указаны все атрибуты, первичные ключи (PK), внешние ключи (FK), альтернативные ключи (AK) и не уникальные индексы (IE). Данная диаграмма использовалась в качестве основы для дальнейшего проектирования физической модели.

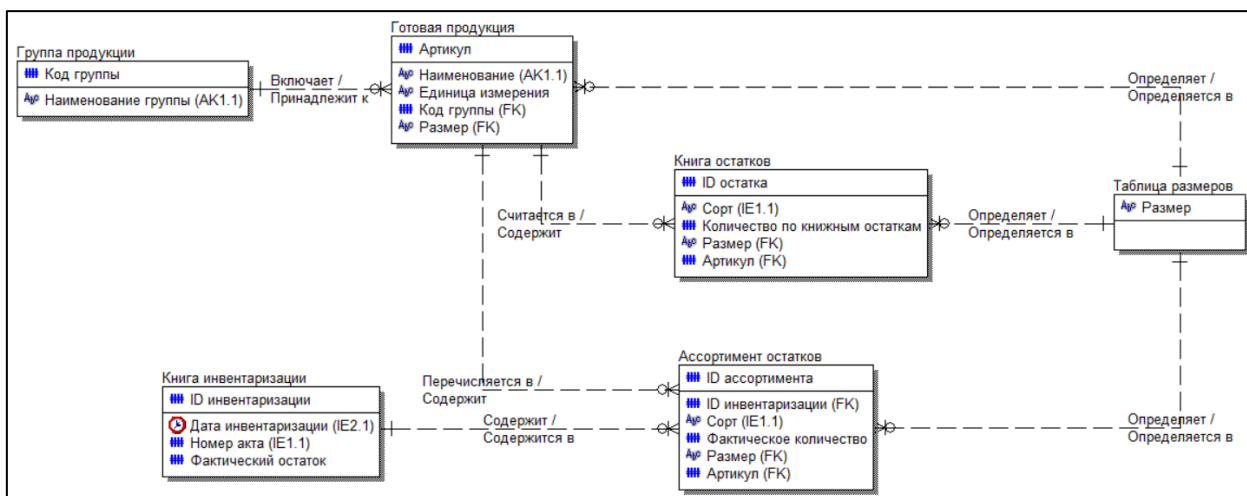


Рисунок 1. Диаграмма сущность-связь (в нотации IE)

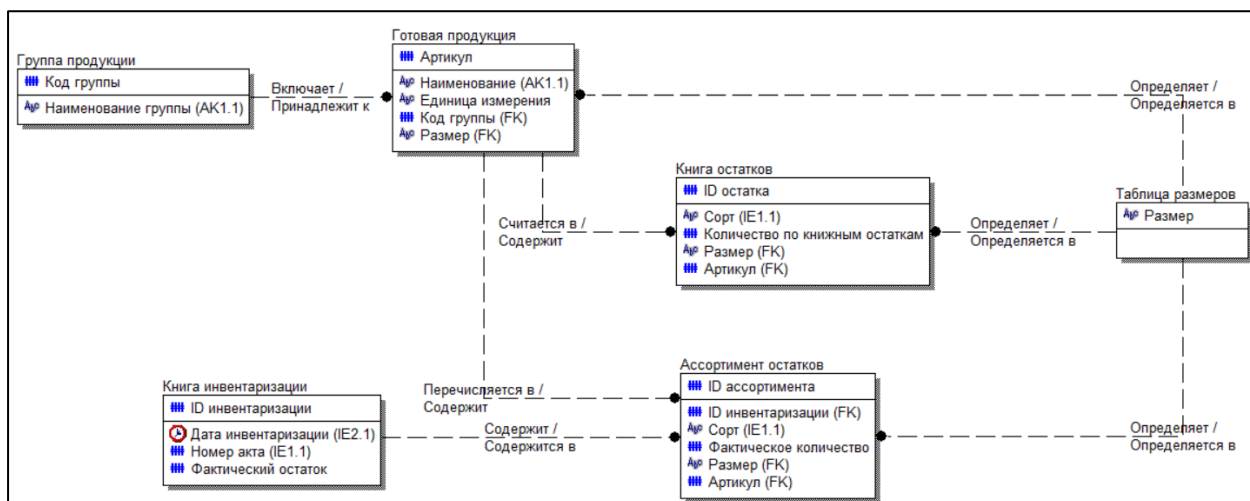


Рисунок 2. Полная атрибутивная модель (в нотации IDEF1X)

Логическая модель включает следующие сущности:

- Группа продукции – справочник, содержащий информацию о группах, к которым относится готовая продукция. Сущность имеет первичный ключ «Код группы» (числовой) и альтернативный ключ «Наименование группы» (строковый), обеспечивающий уникальность названий групп.
- Таблица размеров – справочник размеров продукции. Содержит единственный атрибут «Размер» (строковый), который является первичным ключом.
- Готовая продукция – основная сущность, хранящая данные о выпускаемых изделиях. Первичным ключом служит «Артикул» (числовой). Внешние ключи: «Код группы» (ссылка на сущность «Группа продукции») и «Размер» (ссылка на «Таблица размеров»). Для обеспечения уникальности наименований продукции определён альтернативный ключ «Наименование». Также присутствуют атрибуты «Единицы измерения» (строковый).
- Книга остатков – регистрационная таблица, содержащая сведения о книжных остатках продукции на складе. Первичный ключ – суррогатный «ID остатка» (числовой), введённый для удобства идентификации записей. Внешние ключи: «Артикул» (ссылка на «Готовая продукция») и

«Размер» (ссылка на «Таблица размеров»). Дополнительные атрибуты: «Сорт» (строковый) и «Количество по книжным остаткам» (числовой). Для ускорения поиска по сорту создан не уникальный индекс (IE) «Book_Sort».

- Книга инвентаризации – регистрационная таблица, фиксирующая факты проведения инвентаризаций. Первичный ключ – «ID инвентаризации» (числовой). Атрибуты: «Дата инвентаризации» (дата/время) и «Номер акта» (числовой). Для этих атрибутов созданы индексы (IE) «Data» и «Akt» соответственно, чтобы ускорить выборки по датам и номерам актов.
- Ассортимент остатков – таблица, содержащая данные фактического наличия продукции, выявленного в ходе конкретной инвентаризации. Первичный ключ – «ID ассортимента» (числовой). Внешние ключи: «ID инвентаризации» (ссылка на «Книга инвентаризации»), «Артикул» (ссылка на «Готовая продукция») и «Размер» (ссылка на «Таблица размеров»). Атрибуты: «Сорт» (строковый) и «Фактическое количество» (числовой). Для сорта также создан не уникальный индекс «Assort_Sort».

Все связи между сущностями являются неидентифицирующими (связь «один ко многим»), то есть внешний ключ не входит в состав первичного ключа дочерней сущности. На уровне логической модели для всех связей заданы ограничения: при удалении или обновлении родительской записи (Parent Delete / Parent Update) используется действие RESTRICT – запрет на изменение или удаление, если существуют связанные дочерние записи. Это обеспечивает целостность ссылочных данных. Допустимость Null-значений для внешних ключей не разрешена (Nulls = No), так как каждая дочерняя запись должна ссылаться на существующую родительскую.

Перечень связей с указанием участвующих сущностей, мощности и типа приведён в приложении 1. Перечень дополнительных ключей представлен в таблице 1:

Таблица 1 – Дополнительные ключи логической модели

Имя сущности	Имя ключа	Тип ключа (IE/АК)	Список атрибутов
Группа продукции	Group_Name	АК	Наименование группы
Книга инвентаризации	Akt	IE	Номер акта
Книга инвентаризации	Data	IE	Дата инвентаризации
Готовая продукция	Prod_Name	АК	Наименование
Книга остатков	Book_Sort	IE	Сорт
Ассортимент остатков	Assort_Sort	IE	Сорт

Проверка логической модели на соответствие требованиям нормализации показала, что все сущности находятся в третьей нормальной форме. Во всех таблицах отсутствуют повторяющиеся группы, каждый неключевой атрибут полностью зависит от первичного ключа и не зависит транзитивно от других неключевых атрибутов. Таким образом, разработанная логическая модель является корректной, непротиворечивой и готовой к преобразованию в физическую модель для выбранной СУБД.

2. Описание физической модели данных

На основе разработанной логической модели была создана физическая модель данных в среде проектирования Power Architect. Физическое проектирование учитывало особенности целевой СУБД [2], для которой впоследствии выполнялась генерация скриптов создания объектов. В ходе преобразования логической модели в физическую для каждой сущности были определены конкретные имена таблиц и колонок (на латинице), уточнены типы данных и их размеры, заданы ограничения NOT NULL для обязательных полей, а также установлено свойство автоинкремента для суррогатных первичных ключей. Для всех внешних ключей сохранены правила ссылочной целостности, определённые на уровне логической модели (действия RESTRICT при удалении и обновлении родительской записи). Физическая ER-диаграмма, отражающая все таблицы, колонки, ключи и связи, представлена на рисунке 3.

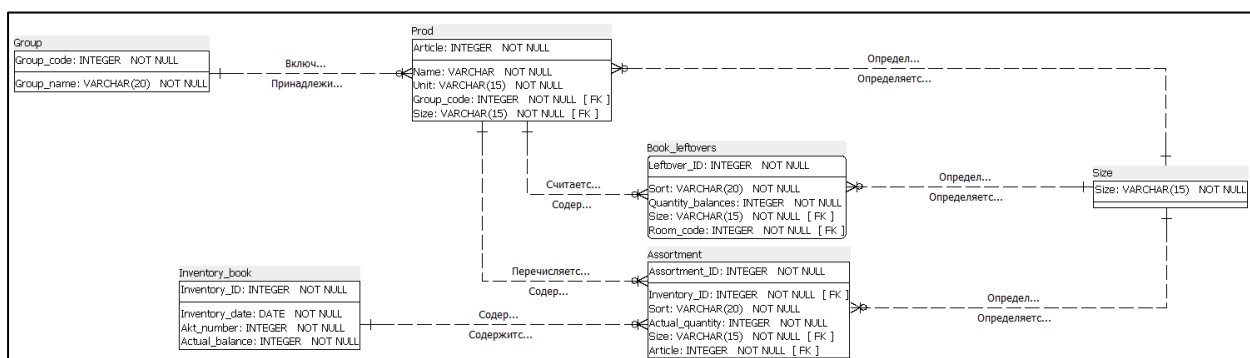


Рисунок 3. Схема физической модели данных

В таблицах 2-7 приведено подробное описание каждой таблицы физической модели с указанием имён атрибутов логической модели, соответствующих им колонок, типов данных и ролей.

Таблица 2 – Группа продукции

Название атрибута	Имя колонки	Тип колонки	Роль
Код группы	Group_code	INTEGER	PK
Наименование группы	Group_name	VARCHAR(20)	AK1.1

Таблица 3 – Таблица размеров

Имя атрибута	Имя колонки	Тип колонки	Роль
Размер	Size	VARCHAR(15)	PK

Таблица 4 – Книга инвентаризации

Имя атрибута	Имя колонки	Тип колонки	Роль
ID инвентаризации	Inventory_ID	INTEGER	PK
Дата инвентаризации	Inventory_date	DATE	IE1.1
Номер акта	Akt_number	INTEGER	IE2.1
Фактический остаток	Actual_balance	INTEGER	

Таблица 5 – Готовая продукция

Имя атрибута	Имя колонки	Тип колонки	Роль
Артикул	Article	INTEGER	PK
Наименование	Name	VARCHAR	AK1.1
Единицы измерения	Unit	VARCHAR(15)	
Размер	Size	VARCHAR(15)	FK
Код группы	Group_code	INTEGER	FK

Таблица 6 – Книга остатков

Имя атрибута	Имя колонки	Тип колонки	Роль
ID остатка	Assortment_ID	INTEGER	PK
Сорт	Sort	VARCHAR(20)	IE1.1
Количество по книжным остаткам	Quantity_balances	INTEGER	
Размер	Size	VARCHAR(15)	FK
Артикул	Article	INTEGER	FK

Таблица 7 – Ассортимент остатков

Имя атрибута	Имя колонки	Тип колонки	Роль
ID ассортимента	Assortment_ID	INTEGER	PK
ID инвентаризации	Inventory_ID	INTEGER	FK
Сорт	Sort	VARCHAR(20)	IE1.1
Фактическое количество	Actual_quantity	INTEGER	
Размер	Size	VARCHAR	FK
Артикул	Article	INTEGER	FK

Все таблицы физической модели спроектированы с учётом требований целостности данных: поля внешних ключей имеют ограничение NOT NULL, типы данных выбраны с учётом характера хранимой информации, а для суррогатных ключей предусмотрена автоматическая генерация значений. Созданные индексы (АК и IE) обеспечивают быстрый доступ при выполнении запросов фильтрации, сортировки и объединения таблиц. Физическая модель полностью соответствует логической модели и готова к реализации в выбранной СУБД.

3. Создание базы данных

Для реализации базы данных была выбрана система управления базами данных PostgreSQL как одна из наиболее распространённых и полнофункциональных СУБД, обеспечивающая надёжное хранение данных, поддержку ссылочной целостности, развитый механизм представлений и триггеров, а также удобные средства администрирования [3]. Администрирование базы данных выполнялось с использованием графической среды pgAdmin, входящей в состав дистрибутива PostgreSQL.

В рамках работы была создана база данных с именем mydatabase. Для логического объединения объектов предметной области в базе данных создана схема prod, в которой размещаются все таблицы, представления, последовательности и триггеры разработанной системы. Скриншот окна свойств созданной базы данных (вкладка «Properties – Settings») приведён на рисунках 4, 5.

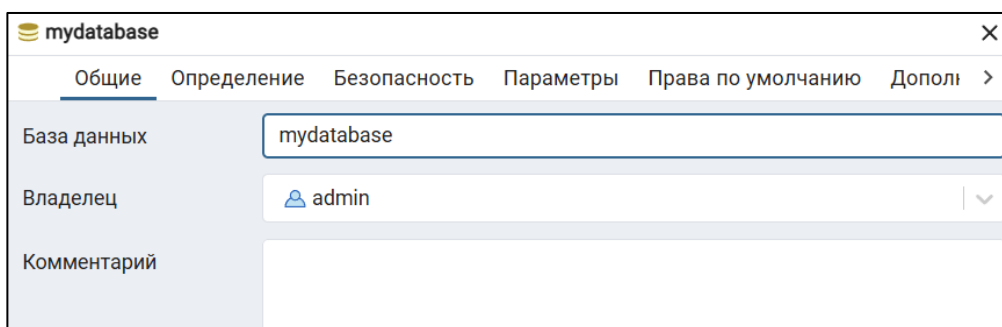


Рисунок 4. Скриншот свойств созданной БД (Общие)

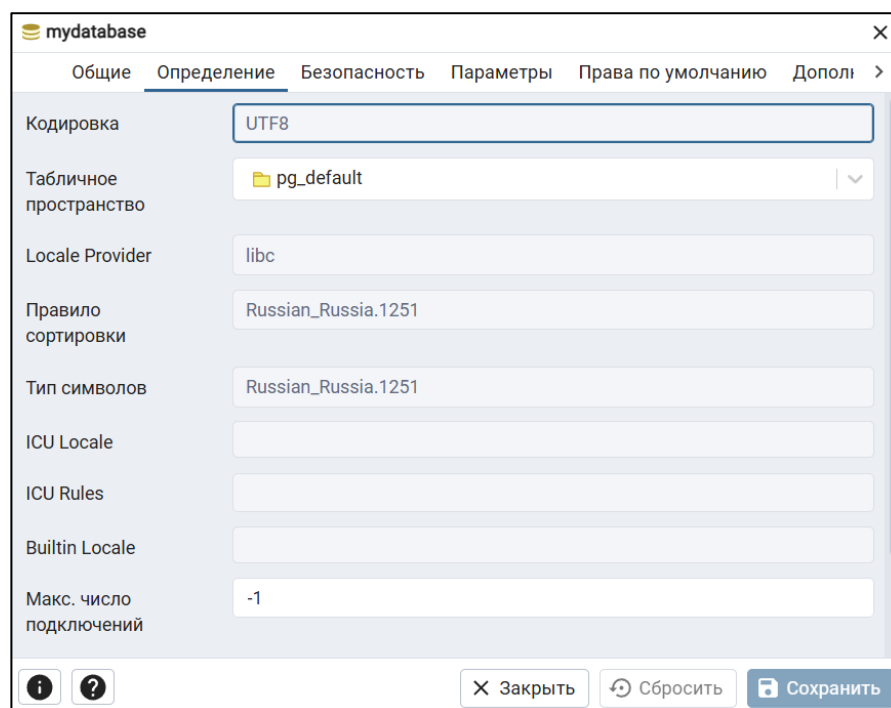


Рисунок 5. Скриншот свойств созданной БД (Определение)

Для разграничения прав доступа в PostgreSQL используется механизм ролей. Роль представляет собой пользователя базы данных, которому назначается определённый набор привилегий на выполнение операций с объектами БД [4]. В рамках курсовой работы была создана роль admin с правами суперпользователя (Superuser). Данная роль является владельцем базы данных, схемы и всех созданных объектов, а также обладает полным набором привилегий для администрирования базы данных. Скриншот узла ролей входа с созданной ролью приведён на рисунке 6, Скриншот параметров (свойств) созданной роли на рисунке 7.

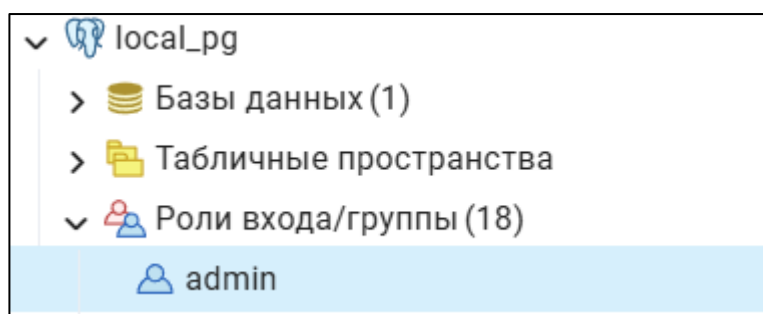


Рисунок 6. Скриншот узла Роли входа с созданной ролью

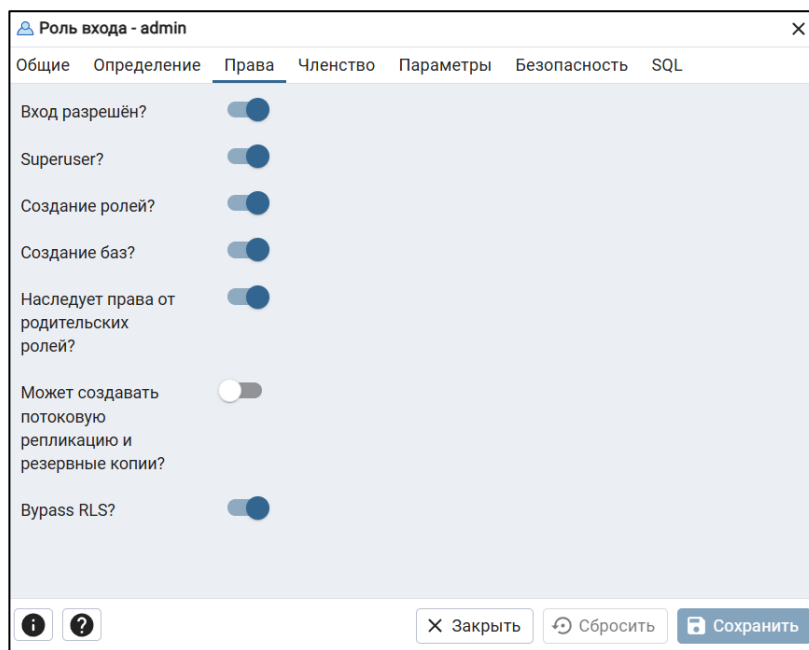


Рисунок 7. Скриншот параметров (свойств) созданной роли

Для реализации физической модели данных в целевой базе данных использовалось средство визуального моделирования SQL Power Architect. На основе разработанной физической модели был сгенерирован DDL-скрипт создания объектов базы данных и затем выполнен в программе pgAdmin. В результате успешного выполнения сценария в схеме prod были созданы все таблицы, последовательности для суррогатных ключей, а также первичные и внешние ключи в соответствии со спроектированной физической моделью. Текст скрипт-файла создания схемы базы данных представлен в Приложении 2. ER-диаграмма созданной базы данных, отображающая структуру таблиц и связи между ними, приведена на рисунке 8.

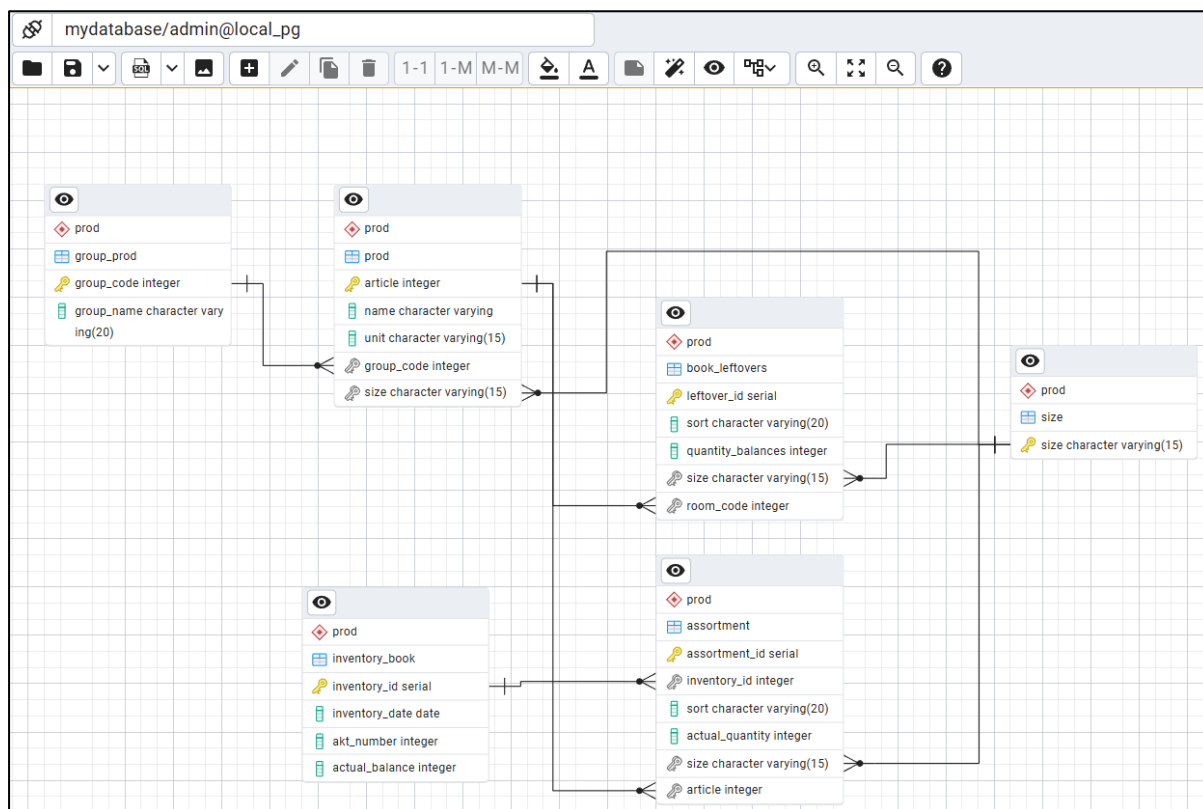


Рисунок 8. Скриншот ERD-диаграммы модели данных

Заполнение созданных таблиц данными выполнялось в следующей последовательности: сначала загружались справочные таблицы, не имеющие внешних ключей (Size, Group_prod), затем таблица Prod, ссылающаяся на справочники, далее таблицы Inventory_book и Book_leftovers, и наконец таблица Assortment, имеющая внешние ключи на все предыдущие. Данные были предварительно подготовлены в виде отдельных листов в формате CSV. Полные тексты CSV-файлов приведены в Приложении 3. Загрузка данных из CSV-файлов в таблицы базы данных выполнялась в среде pgAdmin. Текст скрипта команд загрузки и выгрузки данных представлен в Приложении 4.

В рамках работы для базы данных учёта инвентаризации готовой продукции были созданы два представления, реализующие различные задачи анализа данных [5].

Первое представление – `assortment_view` – предназначено для получения детальной информации об ассортименте инвентаризации. Оно объединяет данные из таблиц Assortment, Inventory_book, Prod, Group_prod и Size. В

представление выводятся: идентификатор ассортимента, дата инвентаризации, номер акта, артикул, наименование продукции, группа продукции, размер, сорт и фактическое количество. Это представление позволяет просматривать полную информацию о каждой позиции инвентаризации без необходимости выполнять многотабличные соединения в каждом запросе. Скриншот кода представления находится на рисунке 9. Результат выполнения запроса на рисунке 10.

 assortment_view_document

Общие

Определение

Код

Безопасность

SQL

1

2

3

4

5

6

7

8

9

10

11

12

```
SELECT a.assortment_id,
      a.sort,
      a.actual_quantity,
      p.name AS product_name,
      s.size,
      ib.inventory_id,
      ib.inventory_date,
      ib.akt_number
FROM prod.assortment a
JOIN prod.prod p ON a.article = p.article
JOIN prod.size s ON a.size::text = s.size::text
JOIN prod.inventory_book ib ON a.inventory_id = ib.inventory_id;
```

Рисунок 9. Код представления

Запрос

История запросов

1

2

```
SELECT * FROM prod.assortment_view_document
WHERE Inventory_ID = 2;
```

Data Output

Сообщения

Notifications

     SQL

Showing rows: 1 to 3  Page No: 1 of 1

	assortment_id integer	sort character varying (20)	actual_quantity integer	product_name character varying	size character varying (15)	inventory_id integer	inventory_date date	akt_number integer
1	22	Металл	12	Стул Металл	50x100	2	2026-01-15	124
2	3	Кожа	10	Диван Кожа	200x200	2	2026-01-15	124
3	4	Клен	5	Кровать Клен	200x200	2	2026-01-15	124

Запрос

История запросов

1

2

```
SELECT * FROM prod.assortment_view_document
WHERE Inventory_ID = 5;
```

Data Output

Сообщения

Notifications

     SQL

Showing rows: 1 to 3  Page No: 1 of 1

	assortment_id integer	sort character varying (20)	actual_quantity integer	product_name character varying	size character varying (15)	inventory_id integer	inventory_date date	akt_number integer
1	9	Ткань	15	Диван Ткань	150x200	5	2026-03-01	127
2	10	Эко-кожа	12	Диван Эко-кожа	200x200	5	2026-03-01	127
3	25	Сосна	10	Полка Сосна	20x40	5	2026-03-01	127

Рисунок 10. Результат выполнения запроса

Второе представление – `group_view_month_summary` – предназначено для получения обобщённых данных по инвентаризациям. Оно группирует записи ассортимента по наименованию группы продукции и по месяцу проведения инвентаризации. Для каждой группы и месяца вычисляется общее фактическое количество продукции, зафиксированное в ходе инвентаризаций за этот месяц. Для форматирования даты использована функция `to_char(inventory_date, 'YYYY-MM')`. Представление позволяет быстро оценить динамику остатков по группам товаров в разрезе времени. Скриншот кода представления находится на рисунке 11. Результат выполнения запроса на рисунке 12.

```
group_view_month_summary
```

Общие	Определение	Код	Безопасность	SQL
1				<code>SELECT g.group_name,</code>
2				<code>to_char(ib.inventory_date::timestamp with time zone, 'YYYY-MM'::text) AS inventory_month,</code>
3				<code>count(a.assortment_id) AS items_count,</code>
4				<code>sum(a.actual_quantity) AS total_quantity</code>
5				<code>FROM prod.assortment a</code>
6				<code>JOIN prod.prod p ON a.article = p.article</code>
7				<code>JOIN prod.group_prod g ON p.group_code = g.group_code</code>
8				<code>JOIN prod.inventory_book ib ON a.inventory_id = ib.inventory_id</code>
9				<code>GROUP BY g.group_name, (to_char(ib.inventory_date::timestamp with time zone, 'YYYY-MM'::text))</code>
10				<code>ORDER BY g.group_name, (to_char(ib.inventory_date::timestamp with time zone, 'YYYY-MM'::text));</code>

Рисунок 11. Код представления

Запрос

История запросов

1

SELECT * FROM prod.group_view_month_summary;

Data Output

Сообщения

Notifications

☰

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	group_name character varying (20)	inventory_month text	items_count bigint	total_quantity bigint
1	Двери	2026-02	3	38
2	Двери	2026-05	1	8
3	Диваны	2026-01	1	10
4	Диваны	2026-02	1	3
5	Диваны	2026-03	2	27
6	Диваны	2026-05	1	8
7	Комоды	2026-04	1	8
8	Кровати	2026-01	2	11
9	Кровати	2026-03	1	8
10	Кровати	2026-05	1	10

Рисунок 12. Результат выполнения запроса

Заключение

В ходе выполнения курсовой работы была спроектирована и реализована база данных для автоматизации учёта готовой продукции и проведения инвентаризаций на складе предприятия. В рамках работы выполнен анализ предметной области, в результате которого определены основные сущности, их атрибуты и взаимосвязи.

Разработана логическая модель данных в третьей нормальной форме, отражающая все необходимые справочники, регистры учёта и документы инвентаризаций. Создана физическая модель для СУБД PostgreSQL, учитывающая особенности типов данных и ограничений целостности.

На основе разработанных моделей реализована база данных. Спроектирована структура таблиц, позволяющая хранить полную информацию о продукции, складских остатках по учётным данным и результатах фактических инвентаризаций. База данных поддерживает ссылочную целостность через механизмы первичных и внешних ключей, что гарантирует согласованность данных при выполнении операций добавления, изменения и удаления записей.

Реализованные средства позволяют получать информацию об излишках и недостатках в разрезе групп продукции, артикулов, размеров и сортов, что даёт возможность своевременно выявлять ошибки учёта и принимать корректирующие управленческие решения.

В результате выполнения курсовой работы создана работоспособная база данных, соответствующая требованиям индивидуального задания. Разработанная информационная система предоставляет основу для дальнейшего развития – добавления аналитических отчётов, интерфейсов для пользователей и интеграции с другими подсистемами предприятия. Полученные навыки проектирования баз данных могут быть применены при создании информационных систем складского учёта и внутреннего контроля на предприятиях различных отраслей.

Список литературы

1. Константинова Л.Ф., Ямбаева О.Б., Дабаева Б.Ц. Роль информационных систем в современном мире и тенденции их развития // Baikal Research Journal. – 2024. – Т. 15, № 3. – С. 1153-1163.
2. Википедия: База данных - https://ru.wikipedia.org/wiki/База_данных?ysclid=mq5had4pz5482657282 (дата обращения: 12.06.2026)
3. Лузанов П.В., Рогов Е.В., Лёвшин И.В. Postgres. Первое знакомство. – 12-е изд., перераб. и доп. – М.: Постгрес Профессиональный, 2026.
4. Документация к PostgreSQL 18.4. – <https://repo.postgrespro.ru/doc/pgsql/18.4/ru/postgres-A4.pdf> (дата обращения: 12.06.2026).
5. Представления, VIEW - <https://sql-academy.org/ru/guide/view?ysclid=mq8jaldfup426731640> (дата обращения: 12.06.2026).

Приложение 1. Связи между сущностями

Таблица П.1 – Перечень связей между сущностями

Связанные сущности	Имя связи	Мощность связи	Тип связи	Допустимость Null-значений	Действия на удаление	Действия на обновление
Группа продукции – Готовая продукция	Включает / Принадлежит к	Один ко многим	Неидентифицирующая	Нет	RESTRICT	RESTRICT
Книга инвентаризации – Ассортимент остатков	Содержит / Содержится в	Один ко многим	Неидентифицирующая	Нет	RESTRICT	RESTRICT
Таблица размеров – Готовая продукция	Определяет / Определяется в	Один ко многим	Неидентифицирующая	Нет	RESTRICT	RESTRICT
Таблица размеров – Книга остатков	Определяет / Определяется в	Один ко многим	Неидентифицирующая	Нет	RESTRICT	RESTRICT
Таблица размеров – Ассортимент остатков	Определяет / Определяется в	Один ко многим	Неидентифицирующая	Нет	RESTRICT	RESTRICT
Готовая продукция – Книга остатков	Считается в / Содержит	Один ко многим	Неидентифицирующая	Нет	RESTRICT	RESTRICT
Готовая продукция – Ассортимент остатков	Перечисляется в / Содержит	Один ко многим	Неидентифицирующая	Нет	RESTRICT	RESTRICT

Приложение 2. Текст скрипт-файла создания схемы БД

```
CREATE TABLE prod.Size (
    Size VARCHAR(15) NOT NULL,
    CONSTRAINT size_pk
PRIMARY KEY (Size)
);
CREATE SEQUENCE
prod.inventory_book_inventory_id_seq;

CREATE TABLE prod.Inventory_book (
    Inventory_ID INTEGER NOT
NULL DEFAULT
nextval('prod.inventory_book_inventory_id_
seq'),
    Inventory_date DATE NOT NULL,
    Akt_number INTEGER NOT NULL,
    Actual_balance INTEGER
DEFAULT 0 NOT NULL,
    CONSTRAINT inventory_book_pk
PRIMARY KEY (Inventory_ID)
);
ALTER SEQUENCE
prod.inventory_book_inventory_id_seq
OWNED BY
prod.Inventory_book.Inventory_ID;

CREATE TABLE prod.Group_prod (
    Group_code INTEGER NOT NULL,
    Group_name VARCHAR(20) NOT NULL,
    CONSTRAINT group_prod_pk
PRIMARY KEY (Group_code)
);
CREATE TABLE prod.Prod (
    Article INTEGER NOT NULL,
    Name VARCHAR NOT NULL,
    Unit VARCHAR(15) DEFAULT
mm NOT NULL,
    Group_code INTEGER NOT
NULL,
    Size VARCHAR(15) NOT NULL,
    CONSTRAINT prod_pk
PRIMARY KEY (Article)
);
CREATE SEQUENCE
prod.assortment_assortment_id_seq;
CREATE TABLE prod.Assortment (
    Assortment_ID INTEGER NOT
NULL DEFAULT
nextval('prod.assortment_assortment_id_seq'
),
    Inventory_ID INTEGER NOT
NULL,
    Sort VARCHAR(20) NOT NULL,
    Actual_quantity INTEGER NOT
NULL,
    Size VARCHAR(15) NOT NULL,
    Article INTEGER NOT NULL,
    CONSTRAINT assortment_pk
PRIMARY KEY (Assortment_ID)
);
ALTER SEQUENCE
prod.assortment_assortment_id_seq OWNED
BY prod.Assortment.Assortment_ID;
CREATE SEQUENCE
prod.book_leftovers_leftover_id_seq;
CREATE TABLE prod.Book_leftovers (
```

```

        Leftover_ID    INTEGER    NOT
NULL                                DEFAULT
nextval('prod.book_leftovers_leftover_id_seq
'),
    Sort VARCHAR(20) NOT NULL,
    Quantity_balances INTEGER NOT NULL,
    Size VARCHAR(15) NOT NULL,
    Room_code INTEGER NOT NULL,
        CONSTRAINT book_leftovers_pk
PRIMARY KEY (Leftover_ID)
);
ALTER                                SEQUENCE
prod.book_leftovers_leftover_id_seq
OWNED                                BY
prod.Book_leftovers.Leftover_ID;
ALTER TABLE prod.Assortment ADD
CONSTRAINT size_assortment_fk
FOREIGN KEY (Size)
REFERENCES prod.Size (Size)
ON DELETE RESTRICT
ON UPDATE RESTRICT
NOT DEFERRABLE;
ALTER TABLE prod.Book_leftovers ADD
CONSTRAINT size_book_leftovers_fk
FOREIGN KEY (Size)
REFERENCES prod.Size (Size)
ON DELETE RESTRICT
ON UPDATE RESTRICT
NOT DEFERRABLE;
ALTER TABLE prod.Prod ADD
CONSTRAINT size_prod_fk
FOREIGN KEY (Size)
REFERENCES prod.Size (Size)
ON DELETE RESTRICT

```

```

ON UPDATE RESTRICT
NOT DEFERRABLE;
ALTER TABLE prod.Assortment ADD
CONSTRAINT
inventory_book_assortment_fk
FOREIGN KEY (Inventory_ID)
REFERENCES prod.Inventory_book
(Inventory_ID)
ON DELETE RESTRICT
ON UPDATE RESTRICT
NOT DEFERRABLE;
ALTER TABLE prod.Prod ADD
CONSTRAINT group_code_prod_fk
FOREIGN KEY (Group_code)
REFERENCES prod.Group_prod
(Group_code)
ON DELETE RESTRICT
ON UPDATE RESTRICT
NOT DEFERRABLE;
ALTER TABLE prod.Book_leftovers ADD
CONSTRAINT prod_book_leftovers_fk
FOREIGN KEY (Room_code)
REFERENCES prod.Prod (Article)
ON DELETE RESTRICT
ON UPDATE RESTRICT
NOT DEFERRABLE;
ALTER TABLE prod.Assortment ADD
CONSTRAINT prod_assortment_fk
FOREIGN KEY (Article)
REFERENCES prod.Prod (Article)
ON DELETE RESTRICT
ON UPDATE RESTRICT
NOT DEFERRABLE;

```

Приложение 3. Тексты файлов для загрузки данных таблиц

Size.csv

Размер

20х40

50х100

60х120

70х140

80х160

100х150

120х180

150х200

200х200

Group_prod.csv

Код группы;Наименование группы

1;Двери

2;Столы

3;Стулья

4;Диваны

5;Кровати

6;Шкафы

7;Тумбы

8;Комоды

9;Полки

Prod.csv

Артикул;Наименование;Единица

измерения;Код группы;Размер

1001;Дверь ПВХ;шт;1;100х150

1002;Дверь Дуб;шт;1;80х160

1003;Дверь Сосна;шт;1;70х140

2001;Стол Дуб;шт;2;50х100

2002;Стол Бук;шт;2;60х120

2003;Стол Ясень;шт;2;70х140

3001;Стул Дуб;шт;3;50х100

3002;Стул Бук;шт;3;60х120

3003;Стул Металл;шт;3;50х100

4001;Диван Кожа;шт;4;200х200

4002;Диван Ткань;шт;4;150х200

4003;Диван Эко-кожа;шт;4;200х200

5001;Кровать Клен;шт;5;200х200

5002;Кровать Сосна;шт;5;150х200

5003;Кровать Дуб;шт;5;200х200

6001;Шкаф Дуб;шт;6;120х180

6002;Шкаф Сосна;шт;6;100х150

7001;Тумба Дуб;шт;7;80х160

8001;Комод Бук;шт;8;100х150

9001;Полка Сосна;шт;9;20х40

Inventory_book.csv

ID инвентаризации;Дата
инвентаризации;Номер акта;Фактический
остаток

1;01.01.2026;123;12
2;15.01.2026;124;23
3;01.02.2026;125;34
4;15.02.2026;126;45
5;01.03.2026;127;56
6;15.03.2026;128;67
7;01.04.2026;129;78
8;15.04.2026;130;89
9;01.05.2026;131;90
10;15.05.2026;132;100

Book_leftovers.csv

ID остатка;Сорт;Количество по книжным
остаткам;Размер;Артикул

1;ПВХ;10;100x150;1001
2;Дуб;20;50x100;3001
3;Кожа;15;200x200;4001
4;Клен;5;200x200;5001
5;Сосна;12;70x140;1003
6;Бук;18;60x120;2002
7;Ясень;8;70x140;2003
8;Ткань;22;150x200;4002
9;Эко-кожа;25;200x200;4003
10;Клен;15;200x200;5002
11;Дуб;30;120x180;6001
12;Сосна;14;100x150;6002
13;Дуб;20;80x160;7001
14;Бук;16;100x150;8001
15;Сосна;25;20x40;9001

Assortment.csv

ID ассортимента;Сорт;Фактическое
количество;ID
инвентаризации;Размер;Артикул

1;Дуб;5;1;50x100;2001
2;Дуб;15;1;50x100;3001
3;Кожа;10;2;200x200;4001
4;Клен;5;2;200x200;5001
5;ПВХ;12;3;100x150;1001
6;Сосна;8;3;70x140;1003
7;Бук;10;4;60x120;2002
8;Ясень;6;4;70x140;2003
9;Ткань;15;5;150x200;4002
10;Эко-кожа;12;5;200x200;4003
11;Клен;8;6;200x200;5002
12;Дуб;20;6;120x180;6001
13;Сосна;10;7;100x150;6002
14;Дуб;12;7;80x160;7001
15;Бук;8;8;100x150;8001
16;Сосна;15;8;20x40;9001
17;ПВХ;8;9;80x160;1002
18;Дуб;12;9;50x100;3001
19;Кожа;8;10;200x200;4001
20;Клен;10;10;200x200;5001
21;Дуб;6;1;200x200;5003
22;Металл;12;2;50x100;3003
23;Кожа;3;3;200x200;4001
24;Дуб;18;4;100x150;1001
25;Сосна;10;5;20x40;9001

Приложение 4. Скрипты команд загрузки и выгрузки данных

Загрузка:

```
SET search_path TO prod;
COPY Size (Size)
FROM 'C:/data_in/Size.csv'
CSV DELIMITER ';' HEADER ENCODING
'WIN1251';
COPY      Group_prod      (Group_code,
Group_name)
FROM 'C:/data_in/Group_prod.csv'
CSV DELIMITER ';' HEADER ENCODING
'WIN1251';
COPY Prod (Article, Name, Unit, Group_code,
Size)
FROM 'C:/data_in/Prod.csv'
CSV DELIMITER ';' HEADER ENCODING
'WIN1251';
COPY      Inventory_book      (Inventory_ID,
Inventory_date, Akt_number, Actual_balance)
FROM 'C:/data_in/Inventory_book.csv'
CSV DELIMITER ';' HEADER ENCODING
'WIN1251';
COPY Book_leftovers (Leftover_ID, Sort,
Quantity_balances, Size, Article)
FROM 'C:/data_in/Book_leftovers.csv'
CSV DELIMITER ';' HEADER ENCODING
'WIN1251';
COPY Assortment (Assortment_ID, Sort,
Actual_quantity, Inventory_ID, Size, Article)
FROM 'C:/data_in/Assortment.csv'
CSV DELIMITER ';' HEADER ENCODING
'WIN1251';
```

Выгрузка:

```
SET search_path TO prod;
COPY Size TO 'C:/data_out/Size.csv'
CSV DELIMITER ';' HEADER ENCODING
'WIN1251';
COPY      Group_prod      TO
'C:/data_out/Group_prod.csv'
CSV DELIMITER ';' HEADER ENCODING
'WIN1251';
COPY Prod TO 'C:/data_out/Prod.csv'
CSV DELIMITER ';' HEADER ENCODING
'WIN1251';
COPY      Inventory_book      TO
'C:/data_out/Inventory_book.csv'
CSV DELIMITER ';' HEADER ENCODING
'WIN1251';
COPY      Book_leftovers      TO
'C:/data_out/Book_leftovers.csv'
CSV DELIMITER ';' HEADER ENCODING
'WIN1251';
COPY      Assortment      TO
'C:/data_out/Assortment.csv'
CSV DELIMITER ';' HEADER ENCODING
'WIN1251';
```